Sokoban AI – planowanie ruchów z uwzględnieniem ograniczeń fizycznych

Sprawozdanie z projektu nr 5

Sztuczna inteligencja w grach i rozszerzonej rzeczywistości

Autorzy: Alicja Borek, Radosław Firlej, Mariusz Wątroba

Streszczenie. Opracowano grę Sokoban w silniku Godot z algorytmem A* i dyskretnym modelem fizycznym współdzielonym z rozgrywką. Uwzględniono masy obiektów, przyczepność robota, tarcie, poślizg, toczenie beczek, drzwi sterowane płytami naciskowymi i jednokierunkowe uskoki. Na 12 planszach porównano trzy heurystyki w dwóch trybach kosztu. W 216 próbach uzyskano 215 poprawnych planów i jedno przekroczenie limitu czasu. W ENERGY heurystyka Terrain-Aware zmniejszyła liczbę rozwinięć w całej kampanii o 35,1% względem Manhattan przy tym samym koszcie planów.

1 Cel projektu i założenia

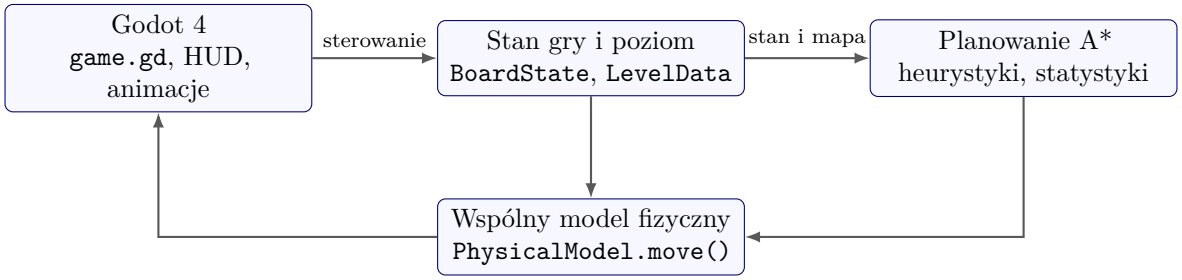
Zadaniem agenta jest umieszczenie skrzyń lub beczek na wszystkich polach docelowych bez naruszania reguł gry; część poziomów zawiera także nadmiarowe obiekty potrzebne do aktywowania płyt. Stan może stać się nierozwiązywalny wskutek zablokowania skrzyni przez przeszkody lub ograniczenia fizyczne. Dlatego celem nie jest wyłącznie znalezienie dowolnej ścieżki, ale również analiza kosztu rozwiązania i sytuacji bez wyjścia (*deadlocków*).

Projekt rozszerzono w dwóch obszarach: (1) model fizyczny, który wpływa na dostępne ruchy i ich koszt, oraz (2) interaktywna gra 2D wykonana w Godot Engine 4, umożliwiająca ręczną rozgrywkę i prezentację działania AI.

Zakres oceny: odsetek rozwiązanych poziomów, długość planu, zużycie energii, liczba rozwiniętych/odkrytych stanów oraz czas obliczeń. Wnioski opieramy na powtarzalnych testach tych samych plansz.

2 Architektura rozwiązania

System rozdziela warstwę wizualną, reprezentację stanu i algorytm od fizycznych reguł ruchu. Dane poziomu (`LevelData`) zawierają kafelki, cele, typy i masy obiektów, siłę robota oraz parametry płyt naciskowych. `BoardState` opisuje pozycję robota i pozycje, masy oraz rodzaje przesuwanych obiektów. Przejścia są obliczane przez `PhysicalModel.move()`, wykorzystywany zarówno przez grę, jak i solver.



Rysunek 1: Uproszczony schemat logiczny architektury. Solver i rozgrywka korzystają z tych samych reguł przejścia.

W HUD można wybrać poziom kampanii, kryterium kosztu (STEPS/ENERGY) i jedną z trzech heurystyk, ustawić maksymalną siłę robota, uruchomić AI i prześledzić wizualizowany plan. Przycisk zatrzymania przerywa odtwarzanie kolejnych ruchów. Kampania obejmuje 12 poziomów z zapisem postępów do pliku `user://progress.cfg`.

3 Model fizyczny i reprezentacja stanu

3.1 Parametry i dopuszczalność ruchu

Model jest dyskretny: $d = 1\,\mathrm{m}$ oznacza długość pola, $g = 9{,}81\,\mathrm{m/s^2}$ przyspieszenie ziemskie, a masa robota wynosi $m_r = 40\,\mathrm{kg}$. Rzeczywista siła, którą robot może przekazać pchanemu obiektowi, zależy nie tylko od silnika, lecz również od przyczepności γ podłoża pod robotem:

$$F_{\mathrm{eff}} = \min(F_{\max},\ \gamma m_r g)\,. \tag{1}$$

Dla skrzyni o masie m_i wymagana siła ruszenia jest równa $F_{\mathrm{start}} = m_i g \mu_s$. Dla beczki opór toczny ρ zastępuje współczynnik tarcia: $F_{\mathrm{start}} = 2 m_i g \rho$. Pchnięcie jest dozwolone, gdy $F_{\mathrm{eff}} \geq F_{\mathrm{start}}$ oraz pole docelowe jest dostępne.

Tabela 1: Podstawowe parametry terenu używane w modelu fizycznym.

Podłoże	μ_s	μ_k	γ	ρ
Drewno i cel	0,35	0,25	0,90	0,015
Lód	0,05	0,03	0,10	0,010
Piasek	0,75	0,65	0,60	0,300

Przyczepność ogranicza siłę na śliskim podłożu, nawet jeśli $F_{\max}$ jest wysokie. Poziomy zawierają ponadto płyty naciskowe otwierające drzwi po ich odpowiednim obciążeniu oraz uskoki, na które można wjechać tylko w kierunku strzałki. Warunki te są sprawdzane w `PhysicalModel.move()` zarówno podczas gry, jak i podczas planowania.

3.2 Energia i poślizg

Koszt zwykłego kroku robota wyraża $W_r = m_r g \mu_k d$, gdzie współczynnik μ_k pochodzi z pola startowego robota. Pchnięcie dodaje pracę przeciw oporowi kinetycznemu obiektu (dla skrzyń $m_i g \mu_k d$, dla beczek $m_i g \rho d$). Jeżeli obiekt wjeżdża na śliskie podłoże, otrzymuje dodatkową energię kinetyczną

$$E_{\mathrm{start}} = \max(0, F_{\mathrm{eff}} - F_{\mathrm{start}})\, d. \tag{2}$$

Na kolejnych polach poślizgu energia maleje o $m_i g(\mu_{k,i} + \mu_{k,i+1})d/2$ (dla beczek zamiast μ_k stosowany jest ρ). Ciało zatrzymuje się po wyczerpaniu energii, przed przeszkodą lub po wjechaniu na pole o większym oporze. Robot także może prześlizgnąć się przez kilka pól lodu, aż odzyska przyczepność lub napotka przeszkodę. Całkowity koszt akcji zwraca model fizyczny; interfejs nie oblicza go ponownie.

W modelu nie symuluje się ciągłego czasu, prędkości ani zderzeń: przesunięcia i zatrzymania są rozpatrywane w pozycjach kafelkowych. Funkcja przejścia zwraca całą trasę obiektu lub robota, dzięki czemu animacje i walidacja planów odpowiadają logice solvera.

4 Algorytm A*, heurystyki i wykrywanie blokad

4.1 Przeszukiwanie stanów

A* wybiera stan o najmniejszej wartości

$$f(n) = g(n) + h(n), \tag{3}$$

gdzie $g(n)$ to koszt dotychczasowej ścieżki, a $h(n)$ oznacza oszacowanie kosztu pozostałego do celu. Rozpatrywane są cztery kierunki ruchu, a ruch może powodować wielopolowy poślizg. Stan opisuje pozycję robota oraz pozycje, masy i rodzaje obiektów; klucz stanu nie zależy od ich kolejności w tablicy. Wyszukiwanie przechowuje najlepszy koszt dla odwiedzonych stanów i rekonstruuje akcje z relacji poprzedników.

Dwa tryby kosztu to `STEPS` (każda akcja kosztuje 1) i `ENERGY` (koszt w dżulach zwracany przez model). Każda heurystyka zwraca oszacowanie w jednostkach wybranego trybu. Kolejka priorytetowa jest kopcem binarnym. Po znalezieniu tańszej drogi do znanego stanu solver może rozwinąć go ponownie.

4.2 Heurystyki

Tabela 2: Heurystyki zastosowane w algorytmie A*.

Wariant	Sposób szacowania kosztu
h_1 Manhattan	Odległość od najbliższego celu podzielona przez maksymalną długość poślizgu i zaokrąglona w górę. W ENERGY każde tak oszacowane pchnięcie kosztuje co najmniej minimalną pracę robota.
h_2 masa i tarcie	Granicę długości pchnięcia wyznacza osobno dla skrzyni i beczki. W ENERGY dolicza minimalną pracę potrzebną do przesunięcia danego obiektu, proporcjonalną do jego masy i najniższego oporu na planszy.
h_3 Terrain-Aware	Wyznacza odległość do najbliższego celu po odwróconych krawędziach uproszczonego grafu pchnięć, metodą Dijkstry startującą jednocześnie z wszystkich celów. Graf zależy od rodzaju i masy obiektu.

Graf h_3 wykorzystuje `PhysicalModel.push_box()`, dzięki czemu uwzględnia ściany, kierunek uskoków, siłę robota, przyczepność i poślizg. Przyjmuje otwarte drzwi oraz dostęp robota do każdego pola, z którego może rozpocząć pchnięcie. Dopuszcza zatrzymanie na dowolnym polu fizycznie możliwej trajektorii, bo w rzeczywistej grze przeszkoda może skrócić poślizg. Krawędź kosztuje jeden ruch albo energię danego pchnięcia. Jeśli za mało obiektów może dotrzeć do celu w uproszczonym grafie, używa skończonego oszacowania h_2.

Wszystkie warianty sumują tyle najmniejszych oszacowań, ile jest celów. Dopuszczenie wspólnego celu dla kilku obiektów, pominięcie kosztu dojścia robota i uproszczenia przejść zapewniają dolne oszacowanie kosztu. Na małych planszach z lodem, beczką zatrzymywaną przez inną skrzynię i drzwiami otwieranymi płytą testy potwierdziły zgodność kosztu h_2 i h_3 z Dijkstrą ($h \equiv 0$) w obu trybach.

4.3 Detekcja deadlocków

Solver wykrywa *statyczne narożniki*: pola poza celami, mające ściany po dwóch prostopadłych stronach. Obiektu wepchniętego do takiego narożnika nie da się już wypchnąć. Algorytm odrzuca stan, gdy liczba obiektów niezablokowanych spada poniżej liczby celów. Nadmiarowe obiekty mogą pozostać poza celami, np. jako obciążniki płyt.

5 Środowisko gry w Godot

Gra wykorzystuje Godot Engine 4.7.2 (GDScript), sterowanie ręczne, HUD, animacje ruchu i poślizgu, wybór kryterium kosztu, regulację $F_{\max}$ oraz podgląd planu AI. Plansze są rysowane programowo, a ruchy odtwarzane przez model fizyczny identyczny jak podczas planowania. Kampania ma 12 kolejno odblokowywanych poziomów, a po ich ukończeniu zapisuje najlepszy wynik i postępy gracza. Nowe mechaniki obejmują beczki, płyty i drzwi oraz jednokierunkowe uskoki.

Wyszukiwanie A* jest synchroniczne i może na wymagających poziomach przejściowo blokować okno. Wyniki planowania obejmują status, liczbę ruchów, energię, czas oraz liczbę rozwiniętych i odkrytych stanów.

6 Metodyka eksperymentów

6.1 Zestaw danych i warunki odtwarzania

Badano 12 poziomów kampanii zapisanych w JSON, od *Pierwszych kroków* do *Wielkiego magazynu*. Plansze zespołu wprowadzają kolejno m.in. poślizg, piasek, ciężkie skrzynie, beczki, uskoki i płyty naciskowe. Każdą kombinację planszy, heurystyki i trybu kosztu uruchomiono trzy razy od tego samego stanu początkowego: $12 \cdot 3 \cdot 2 \cdot 3 = 216$ prób. Wyszukiwania wykonywano kolejno w jednym procesie Godota bez okna. Między powtórzeniami rotowano kolejność heurystyk i zmieniano kolejność trybów kosztu.

Użyto Godot 4.7.2 stable na Windows (kompilacja 26200), procesorze Intel Core i5-12500H i ok. 15,7 GiB pamięci fizycznej. Każda próba korzystała z siły robota zapisanej w JSON (300 N na wszystkich planszach) i limitów gry: 100 000 rozwinięć oraz 30 000 ms wyszukiwania. Wykrywanie blokad narożnych było włączone w każdej próbie.

Pomiar obejmował utworzenie heurystyki i pracę solvera, w tym budowę grafu h_3. Wczytanie poziomu i odtwarzanie planu odbywały się poza pomiarem. Dla każdej konfiguracji raportowano medianę czasu udanych prób oraz przedział min–max w CSV. Liczba odkrytych stanów oznacza liczbę różnych kluczy stanu w mapie najlepszych kosztów. Licznik rozwinięć uwzględnia także powtórne rozwinięcia tego samego stanu.

6.2 Porównania i weryfikacja

- **Porównanie heurystyk.** Na każdej planszy i dla każdego trybu porównano status, koszt optymalizowany, liczbę rozwiniętych i odkrytych stanów oraz czas dla h_1, h_2 i h_3.
- **Porównanie kryteriów kosztu.** Dla planów wyznaczonych z h_1 porównano liczbę ruchów i rzeczywiste zużycie energii w STEPS i ENERGY. Porównanie pokazuje wpływ wyboru funkcji celu przy stałej heurystyce.

Każdy znaleziony plan ponownie wykonano przez `PhysicalModel.move()` od stanu początkowego. Sprawdzono legalność wszystkich akcji, zajęcie wszystkich celów, długość planu, sumaryczną energię i zgodność z optymalizowanym kosztem. Status `time_limit` lub `state_limit` pozostawia rozwiązywalność planszy nieustaloną.

7 Wyniki i interpretacja

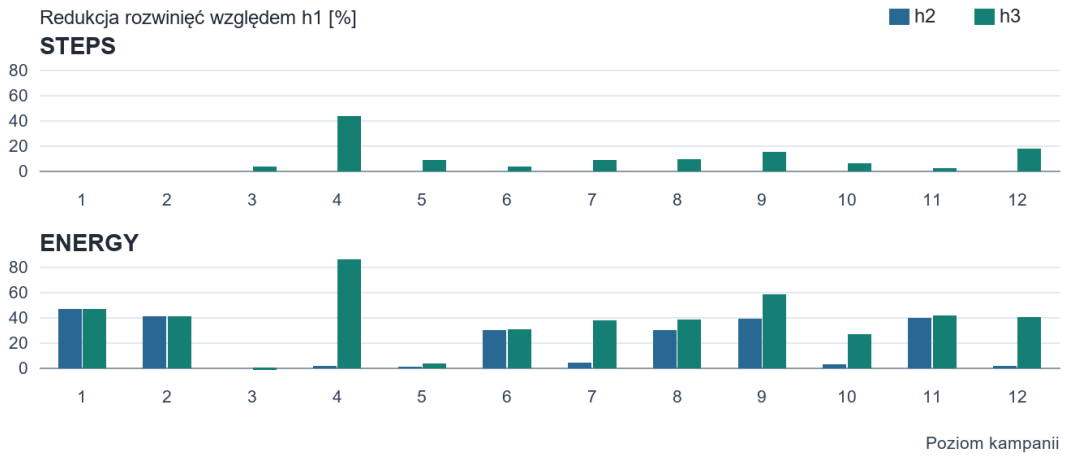
7.1 Porównanie heurystyk

Każda heurystyka rozwiązała wszystkie 12 poziomów w obu trybach przynajmniej raz. Odtworzenie potwierdziło poprawność wszystkich 215 znalezionych planów. Jedno uruchomienie h_2 w ENERGY na poziomie 12 osiągnęło limit czasu po 94 256 rozwinięciach. Pozostałe dwie próby tej konfiguracji zakończyły się rozwiązaniem. W udanych próbach wszystkie trzy heurystyki uzyskały na danej planszy tę samą minimalną liczbę ruchów w STEPS i ten sam minimalny koszt energii w ENERGY.

Tabela 3: Wyniki kampanii według trybu i heurystyki. Stany zsumowano po 12 planszach, po jednej wartości na planszę. Czas to suma 12 median z udanych prób.

Tryb	Heurystyka	Sukcesy	Rozwinięcia	Odkryte stany	Czas [s]
STEPS	h_1	36/36	217 690	248 401	34,08
STEPS	h_2	36/36	217 690	248 401	35,04
STEPS	h_3	36/36	197 302	225 503	32,37
ENERGY	h_1	36/36	280 371	303 277	43,92
ENERGY	h_2	35/36	237 184	259 402	38,60
ENERGY	h_3	36/36	181 903	188 197	29,26

W trybie STEPS liczba rozwinięć dla h_2 była na tych planszach identyczna jak dla h_1. Heurystyka h_3 zmniejszyła łączną liczbę rozwinięć w tym trybie o 9,4%. W ENERGY redukcja wyniosła 15,4% dla h_2 i 35,1% dla h_3. Na *Lodowisku* w ENERGY h_3 rozwinęła o 101 stanów więcej niż h_1, więc korzyść zależy od planszy i kolejności rozwijania stanów.



Rysunek 2: Zmiana liczby rozwinięć względem h_1 na poszczególnych planszach. Słupek dodatni oznacza redukcję, a ujemny wzrost liczby rozwinięć.

Na *Wielkim magazynie* w ENERGY h_3 zmniejszyła liczbę rozwinięć z 98 730 do 59 039, czyli o 40,2%. Mediana czasu spadła z 16,86 do 10,07 s. Koszt planu dla wszystkich trzech heurystyk wynosił 7340,57 J. Na pierwszej planszy w STEPS wszystkie warianty rozwinęły po 41 stanów, a h_3 potrzebowała 5,49 ms wobec 3,95 ms dla h_1. Przy małej przestrzeni stanów przygotowanie grafu może więc zwiększyć czas wyszukiwania.

Tabela 4: Wyniki dla poziomu 12, *Wielki magazyn*. Wartości dla h_2 w ENERGY pochodzą z dwóch udanych prób.

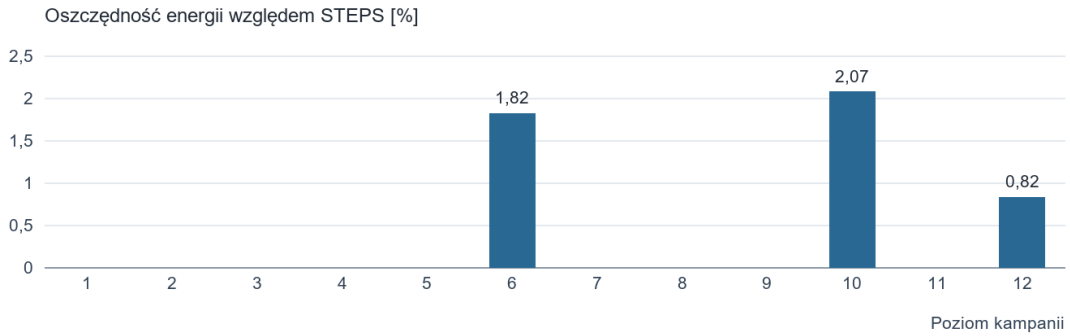
Tryb	Heurystyka	Rozwinięcia	Odkryte stany	Mediana [s]
STEPS	h_1	78 812	90 751	13,28
STEPS	h_2	78 812	90 751	13,60
STEPS	h_3	64 912	74 834	11,62
ENERGY	h_1	98 730	105 663	16,86
ENERGY	h_2	97 187	104 120	16,88
ENERGY	h_3	59 039	59 821	10,07

7.2 Wpływ funkcji kosztu

Na dziewięciu poziomach plany wyznaczone z h_1 w obu trybach miały tę samą długość i energię. Na trzech poziomach ENERGY zmniejszyło zużycie energii o 0,82–2,07%. W *Ciężarowcu* i *Wielkim magazynie* wymagało to dodatkowych ruchów. W *Lodowych beczkach* oba plany liczyły po 31 ruchów i 7 pchnięć, lecz miały inny przebieg i koszt energetyczny.

Tabela 5: Poziomy, na których ENERGY zmniejszyło zużycie energii względem STEPS dla h_1.

Poziom	Ruchy S/E	STEPS [J]	ENERGY [J]	Zysk [J]	Zysk [%]
6. Ciężarowiec	34/36	6 481,96	6 364,24	117,72	1,82
10. Lodowe beczki	31/31	4 167,03	4 080,71	86,33	2,07
12. Wielki magazyn	52/55	7 401,58	7 340,57	61,01	0,82



Rysunek 3: Procentowa oszczędność energii planu ENERGY względem planu STEPS przy heurystyce h_1.

8 Wnioski i ograniczenia

1. Wspólny model ruchu zapewnił zgodność planowania i odtwarzania. Wszystkie uzyskane plany doprowadziły do rozwiązania przy deklarowanej energii i długości.
2. h_3 dała największą łączną redukcję rozwinięć w obu trybach. W ENERGY h_2 poprawiła wynik względem Manhattan już przez uwzględnienie minimalnej pracy potrzebnej do pchania obiektu.
3. Wybór kryterium zmieniał plan. Na dwóch poziomach dodatkowe ruchy pozwoliły uzyskać mniejszy koszt energetyczny, a na jednym poziomie koszt zmalał przy tej samej długości planu.

Zakres pomiarów obejmuje 12 plansz przygotowanych przez zespół, jeden komputer i trzy powtórzenia na konfigurację. Czasy zależą od sprzętu i obciążenia systemu. Model fizyczny jest dyskretny, a detekcja blokad w solverze obejmuje narożniki. Wyszukiwanie na głównym wątku może chwilowo blokować interfejs. Kolejne badania mogą objąć szerszy zestaw plansz, porównanie wariantów detekcji blokad, wpływ zmiany $F_{\max}$ oraz planowanie w osobnym wątku.

9 Odtwarzalność, kod i podział prac

Repozytorium zespołu: `https://github.com/Man1ek27/Sokoban-ai`. Pomiar wykonano na lokalnym kodzie opartym na commicie `2fef64ff0088`, z implementacją h_2 i h_3 dołączoną do projektu. Kolumna `code_sha256` w danych surowych identyfikuje dokładny zestaw plików GDScript i JSON użyty podczas pomiaru.

Grę uruchamia się przez otwarcie `project.godot` w Godot 4.7.2 i naciśnięcie F5. Polecenia testów i benchmarku, wykonywane z katalogu projektu, są podane poniżej. Import jest potrzebny przy pierwszym uruchomieniu od zera.

```
godot --headless --editor --path . --import
godot --headless --path . --script res://tests/physics_test.gd
godot --headless --path . --script res://tests/game_integration_test.gd
godot --headless --path . --script res://sprawozdanie/benchmark.gd
```

Testy fizyki i solvera obejmowały 97 sprawdzeń, a test integracji gry 80. Wszystkie przeszły. Dane surowe zawierają 216 wierszy w `benchmark_raw.csv`. Agregaty dla 72 konfiguracji są w `benchmark_summary.csv`. Wykresy, tabele i agregaty można odtworzyć poleceniem `python sprawozdanie/aggregate_benchmark.py` (Python 3, Pillow).

Tabela 6: Podział odpowiedzialności w zespole.

Osoba	Zakres prac
Radosław Firlej	A*, reprezentacja stanu, trzy heurystyki, wykrywanie blokad narożnych, uruchomienie solvera bez okna i porównanie heurystyk.
Alicja Borek	Środowisko Godot, HUD, animacje, integracja AI z rozgrywką, konfiguracja siły i wizualizacja planu.
Mariusz Wątroba	Model fizyczny, przygotowanie kampanii poziomów, nowe mechaniki oraz testy fizyczne.

Źródła

1. Materiały do przedmiotu *Sztuczna inteligencja w grach i rozszerzonej rzeczywistości*, 2026.
2. Repozytorium zespołu: Sokoban AI (GitHub), stan z 12.11.2026.
3. Wyniki benchmarku z 12.11.2026: `benchmark_raw.csv` oraz `benchmark_summary.csv`.